Rapid Object Detection using a Boosted Cascade of Simple Features

Paul Viola and Michael Jones · CVPR 2001

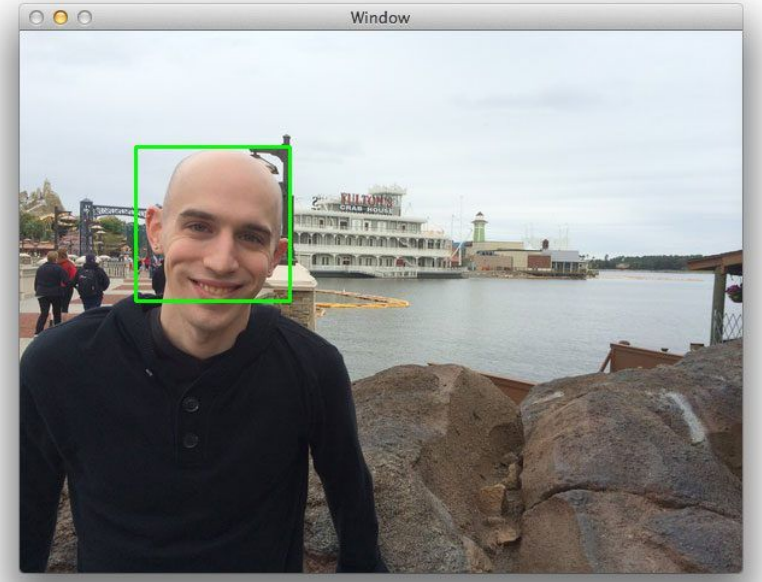
Presented by Uday

The problem: face detection

- Input: a grey-scale image. Output: If the image has any faces, and if so, where
- Faces vary in scale, position, lighting and pose
- Many locations and scales have to be checked
- Goal: high detection rate, few false positives, and speed/efficiency

Subwindows

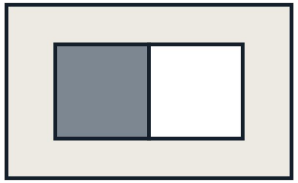
A sub-window is a small rectangular part of a larger image. Looking at an image one sub-window at a time means sliding such a “patch” across the image at different positions and sizes, so each patch can be examined on its own for whether it contains useful information.



Haar-like rectangle features

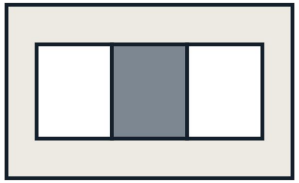
Two-rectangle

Difference between the sums of two adjacent regions of the same size and shape



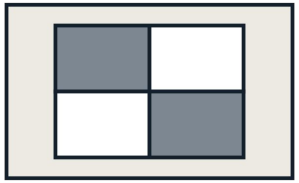
Three-rectangle

Sum of the two outside rectangles subtracted from the sum of the center rectangle



Four-rectangle

Difference between the sums of the two diagonal pairs of rectangles



Feature value = sum of pixel intensities in the grey rectangles – sum of pixel intensities in the white rectangles. They respond to edges, bars and other simple structure.

These Haar-like features are computed for every possible rectangle in a subwindow for this algorithm.

The integral image

Definition

ii(x, y) is the sum of all pixel intensities above and to the left of (x, y), inclusive:

$ii(x, y) = \Sigma\, i(x', y')$ over $x' \le x, y' \le y$

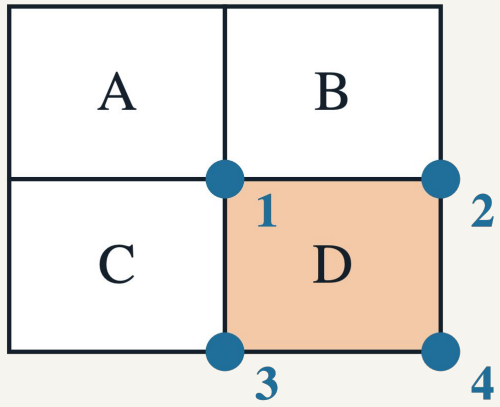
One pass over the image

Keep a cumulative row sum s(x, y):

$s(x, y) = s(x, y-1) + i(x, y)$

$ii(x, y) = ii(x-1, y) + s(x, y)$

with $s(x, -1) = 0$ and $ii(-1, y) = 0$



Sum(D) = ii(4) + ii(1) − ii(2) − ii(3)

Any rectangle sum takes four array references.

Using the integral image

Feature	Rectangles	Array references
Two-rectangle	2	6
Three-rectangle	3	8
Four-rectangle	4	9

Adjacent rectangles share corners, so fewer than 4 references per rectangle are needed.

- Constant time at any scale or location
- Richer features exist (e.g. steerable filters), but they cost far more to compute
- Rectangle features are coarse: only vertical, horizontal and diagonal orientations
- Their efficiency compensates for the limited flexibility

The problem: too many features

576
pixels in a 24×24 detection window

180,000+
rectangle features in the same window (the set is overcomplete)

- Each feature is cheap, but computing the complete set is prohibitively expensive
- Hypothesis: a very small number of features can be combined into an effective classifier
- The challenge is quickly and efficiently finding them

Can use AdaBoost as part of the solution

- Given example images $(x_1, y_1), \ldots, (x_n, y_n)$ where $y_i = 0, 1$ for negative and positive examples respectively.
- Initialize weights $w_{1,i} = \frac{1}{2m}, \frac{1}{2l}$ for $y_i = 0, 1$ respectively, where m and l are the number of negatives and positives respectively.
- For $t = 1, \ldots, T$:
  1. Normalize the weights,
     $$w_{t,i} \leftarrow \frac{w_{t,i}}{\sum_{j=1}^{n} w_{t,j}}$$
     so that w_t is a probability distribution.
  2. For each feature, j, train a classifier h_j which is restricted to using a single feature. The error is evaluated with respect to w_t, $\epsilon_j = \sum_i w_i \left| h_j(x_i) - y_i \right|$.
  3. Choose the classifier, h_t, with the lowest error ϵ_t.
  4. Update the weights:
     $$w_{t+1,i} = w_{t,i} \beta_t^{1-e_i}$$
     where $e_i = 0$ if example x_i is classified correctly, $e_i = 1$ otherwise, and $\beta_t = \frac{\epsilon_t}{1-\epsilon_t}$.
- The final strong classifier is:
  $$h(x) = \begin{cases} 1 & \sum_{t=1}^{T} \alpha_t h_t(x) \geq \frac{1}{2} \sum_{t=1}^{T} \alpha_t \\ 0 & \text{otherwise} \end{cases}$$
  where $\alpha_t = \log \frac{1}{\beta_t}$

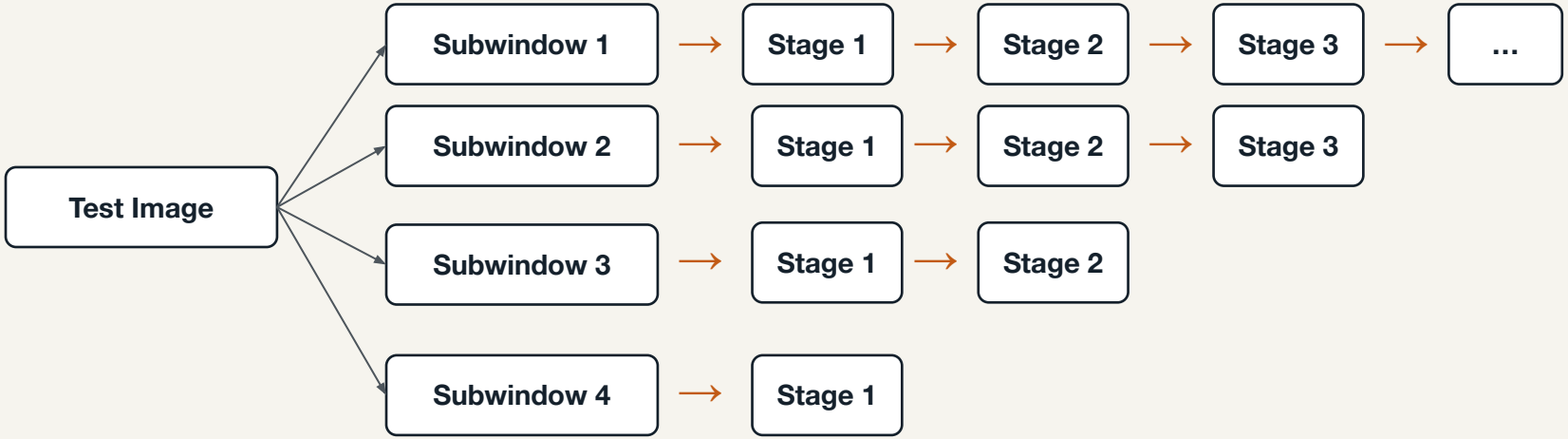
Weak classifier $h_j(x)$ consists of a feature f_j, a threshold θ_j, and a parity p_j indicating the direction of the inequality sign

$$h_j(x) = \begin{cases} 1 & \text{if } p_j f_j(x) < p_j \theta_j \\ 0 & \text{otherwise} \end{cases}$$

The attentional cascade: the idea

Within any image, an overwhelming majority of sub-windows are **not** useful.

- Stages constructed by training classifiers and then adjusting the threshold to minimize false negatives on validation set
- Reject as many negatives as possible, as early as possible, with very little computation
- Spend more computation only on promising subwindows



Training the cascade

- Each stage adds features until its target detection and false positive rates are met, measured on a validation set
- Stages are added until the overall targets for false positives and detection are met
- Later stages are trained only on examples that pass all earlier stages, so they face harder negatives

Per-stage targets

A minimum reduction in false positives and a maximum decrease in detection

More features

Higher detection and fewer false positives, but more computation

The algorithm in pseudo-code

Training

```
repeat until overall FP and
  detection targets are met:
  stage = empty classifier
  while targets missed on
    the validation set:
    add 1 feature (AdaBoost)
    lower threshold: keep detection
  add stage to cascade
  next negatives = false
    positives of cascade so far
```

Detection

```
for each sub-window:
  for each stage:
    if subwindow fails stage:
      reject, stop
  report a detection
```

Results and comparisons

Detection rate at N false positives	10	31	50	65	78	95	167
Viola-Jones	76.1%	88.4%	91.4%	92.0%	92.1%	92.9%	93.9%
Viola-Jones (voting)	81.1%	89.7%	92.1%	93.1%	93.1%	93.2%	93.7%
Rowley-Baluja-Kanade	83.2%	86.0%	–	–	–	89.2%	90.1%
Schneiderman-Kanade	–	–	–	94.4%	–	–	–
Roth-Yang-Ahuja	–	–	–	–	94.8%	–	–

38 stages

6,061 features in total; about 10 evaluated per sub-window

About 15× faster

0.067 s per 384×288 image; 15× faster than Rowley et al., 600× faster than Schneiderman-Kanade

MIT+CMU test set

130 images, 507 faces; accuracy comparable to the best earlier systems

Final Additions to Algorithm

- Image subwindows were variance normalized (i.e. standardization) to remove the effect of variable lighting
    - Can be efficiently done while scanning the image instead of preprocessing the image
- The detector was also scaled up (increasing the size of the subwindow and its' Haar-like rectangle features by a scale factor) to detect larger facial patterns
- Detection of face - overlapping subwindow detections, with corners of that detection region the average of all the overlapping subwindow detections there

Bhavik Jikadara · Updated 3 years ago

267 Code Download

Cats and Dogs Classification Dataset

The Cat and Dog Classification dataset is a standard computer vision dataset.



About Dataset

The Cat and Dog Classification dataset is a standard computer vision dataset that involves classifying photos as either containing a dog or a cat. This dataset is provided as a subset of photos from a much larger dataset of approximately 25 thousands.

The dataset contains **24,998 images**, split into **12,499 Cat images** and **12,499 Dog images**. The training images are divided equally between cat and dog images, while the test images are not labeled. This allows users to evaluate their models on unseen data.

Usability

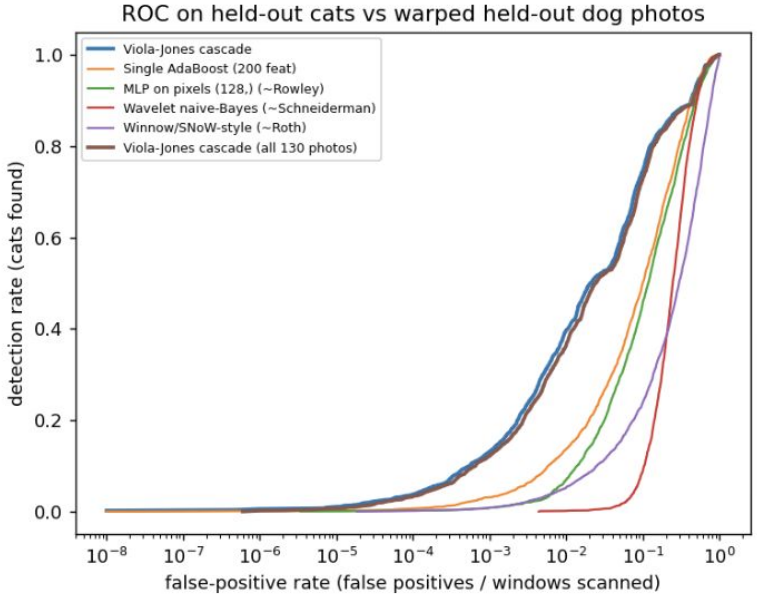
8.13

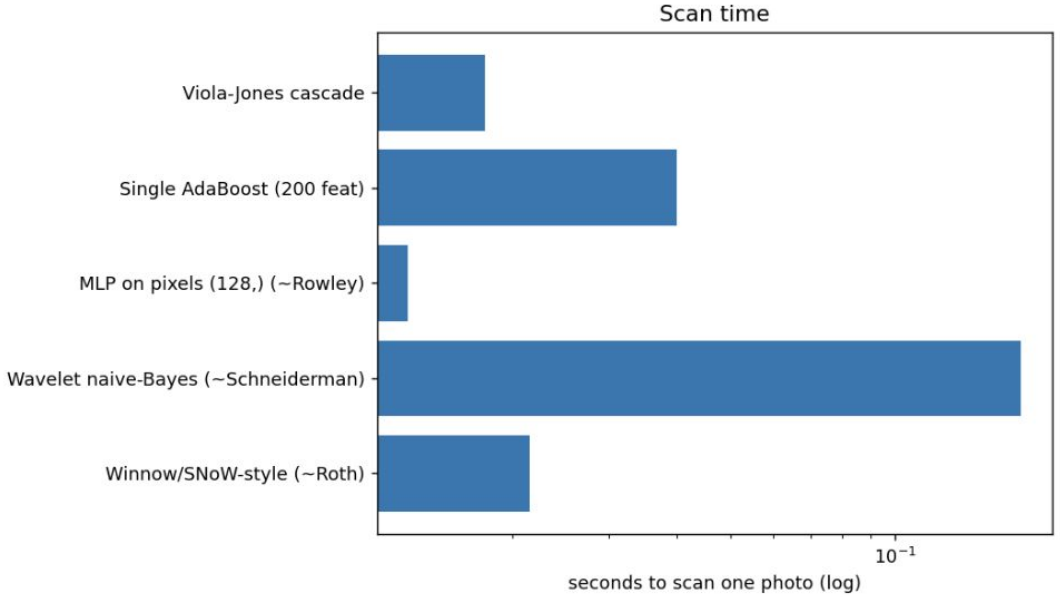
License

Apache 2.0

Expected update frequency

Never


ROC on held-out cats vs warped held-out dog photos
Viola-Jones cascade
Single AdaBoost (200 feat)
MLP on pixels (128,) (~Rowley)
Wavelet naive-Bayes (~Schneiderman)
Winnow/SNoW-style (~Roth)
Viola-Jones cascade (all 130 photos)
detection rate (cats found)
1.0
0.8
0.6
0.4
0.2
0.0
10^{-8} 10^{-7} 10^{-6} 10^{-5} 10^{-4} 10^{-3} 10^{-2} 10^{-1} 10^{0}
false-positive rate (false positives / windows scanned)


Scan time
Viola-Jones cascade
Single AdaBoost (200 feat)
MLP on pixels (128,) (~Rowley)
Wavelet naive-Bayes (~Schneiderman)
Winnow/SNoW-style (~Roth)
10^{-1}
seconds to scan one photo (log)